FILE MANAGEMENT SUBSYSTEM

What is a file in Linux?

In Linux, a file is a container for storing data, but it also serves as an abstraction for interacting with hardware, processes, and network communication.

The classic UNIX philosophy: “Everything is a file”.

The three main types of files in Linux:

1. Regular Files (or) Normal Files
2. Directory Files
3. Special Files

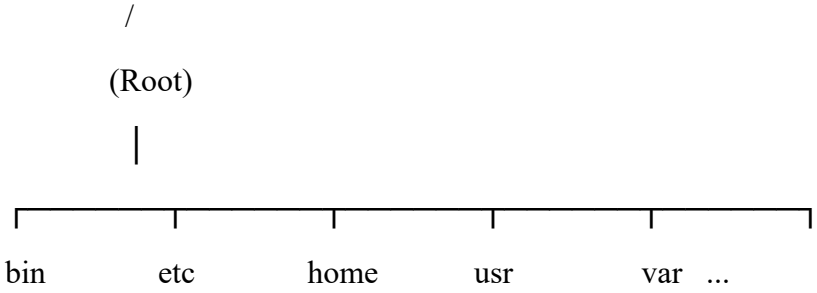
1. **Regular files:** common files that contain standard data, text, or program instructions
    i. Text files
    ii. Binary files
2. **Directory files:** files that act as a container to organize other files hierarchically.
3. **Special files:** files that act as an interface between OS’s software layer and system’s hardware. These files are stored in /dev folder.
    i. Device files
    ii. IPC objects

File System Hierarchy

The **File System Hierarchy Standard (FHS)** defines the structure of the Linux operating system.

The Linux OS can be imagined as a tree.

The Root Directory (/) is the starting point of the entire Linux file system. All the files, directories, and storage devices branch out from this single directory tree.



```
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── sys
├── tmp
├── usr
└── var
```

/bin: contains essential command-line programs needed by all users

/boot: contains files required to boot Linux

/dev: contains device files of physical/virtual hardware devices

/etc: contains host-specific system-wide configuration files

/lib: shared library files required by binaries in /bin and /sbin to boot the system and run the commands

/proc: virtual filesystem exposing kernel and process information

/root: dedicated home directory for the root(administrative) user

/sys: virtual filesystem providing information about device drivers, kernel modules and hardware devices

File Permissions

Linux file permissions control who can read, write, or execute a file or directory. They are essential for system security.

Permissions are assigned to three distinct categories of users:

1. User (u): *the individual account that owns the file*
2. Group (g): *a collection of users who share access rights*
3. Others (o): *every other account on the system*

Types of permissions:

1. Read
2. Write
3. Execute

Permissions are frequently represented as 3-digit octal numbers.

Commands to manage file security, ownership, and access rights:

- Chmod (*change mode)*
- Chown *(change owner)*
- Chgrp *(change group)*

umask:

umask(user mask) is a special shell feature that sets the default permissions for newly created files and directories.

It acts as a security filter that subtracts or blocks permissions from a baseline system maximum.

- Default directory baseline: 777 (*full rwx permissions for UGO)*
- Default file baseline: 666 (*full rw permissions. No execution rights to new files)*

Math Examples

If your system's umask is set to **022** (a common default), the creation calculation looks like this:

- **For Directories:**
  777 (Full Baseline)
  - 022 (umask)
  = 755 (Final Permission: rwxr-xr-x)
- **For Files:**
  666 (Full Baseline)
  - 022 (umask)
  = 644 (Final Permission: rw-r--r--)

Common file management operations:

1. Listing Files: (*lists all files and subdirectories)*
   $ ls
   $ ls -l
   $ ls -la
   $ ls -lh

2. Creating Files: (*creates a new blank file)*
   $ touch <filename>
3. Displaying file contents: (*display the contents of file in terminal)*
   $ cat <filename>
4. Copying a file: (*copies the contents of the source file to a new file)*
   $ cp source/filename destination/
5. Moving a file
   $ mv source/filename destination/
6. Deleting a file
   $ rm <filename>

Basic I/O calls

These are the calls required to access the files present if physical memory or virtual memory.

The **five basic I/O system calls** in Linux are open(), close(), read(), write(), and lseek(). These lower-level kernel calls manipulate files, devices, and sockets using an integer identifier called a **file descriptor (FD)**.

The 5 Core I/O System Calls

1. open()

Opens an existing file or creates a new one, returning a unique file descriptor.

- **Syntax**: int open(const char *pathname, int flags, mode_t mode);
- **Common Flags**: O_RDONLY (read-only), O_WRONLY (write-only), O_RDWR (read/write), and O_CREAT (create file if missing).

2. close()

Closes an active file descriptor, freeing up the resource for other processes.

- **Syntax**: int close(int fd);
- **Return**: Returns 0 on success or -1 if an error occurs.

3. read()

Reads a specified amount of data from a file descriptor into a buffer array.

- **Syntax**: ssize_t read(int fd, void *buf, size_t count);
- **Return**: Returns the number of bytes read, 0 at End-of-File (EOF), or -1 on error.

4. write()

Writes data out from a buffer array directly to a file descriptor.

- **Syntax**: ssize_t write(int fd, const void *buf, size_t count);
- **Return**: Returns the number of bytes written, or -1 on error.

5. lseek()

Reposition the read/write file offset (cursor position) inside an open file.

- **Syntax**: off_t lseek(int fd, off_t offset, int whence);
- **Directives**: SEEK_SET (start of file), SEEK_CUR (current position), or SEEK_END (end of file)